

DOCUMENTAÇÃO TÉCNICA — V1.0

Testes Automatizados **REST** Suite Interna

SCC_Testes & Dynatrace

Data: 09/06/2026 · **Tecnologias:** Java 17 · JUnit 5 · REST Assured · ExtentReports

Cobertura: 8 testes automatizados · **Classes:** SCC_Testes, Dynatrace · **Execução:** mvn test

→ SuiteInterno

Sumário

1. Visão Geral	3
2. Arquitetura e Infraestrutura	3
2.1 Suite de Testes (SuiteInterno)	4
2.2 Configuração (ConfigLoader)	4
2.3 ExtentReportExtension	4
3. SCC_Teste	5
3.1 test_PA_ListarTAGsPossiveis	5
3.2 test_GerarSenha	6
3.3 test_GerarSenhaComPrioridade	6
3.4 test_ChamarProximo	7
4. Dynatrace	8
4.1 test_ValidarParametrosServico	8
4.2 test_ValidarListarTMEDia	9
4.3 test_ValidarMaiorEspera	10
4.4 test_ValidarObterQtdEsperaV2	10
5. Classes Utilitárias	11
5.1 Metodos_Rest	11
5.2 Servico_Parametros	13

1. Visão Geral

Este documento descreve a **Suite Interna** de testes automatizados REST, composta pelas classes `SCC_Testes` e `Dynatrace`. Os testes validam endpoints de um sistema de gerenciamento de filas e atendimento — cobrindo geração de senhas, chamada de próximo, consulta de TME (Tempo Médio de Espera), maior espera em fila e parâmetros de configuração do serviço.

Resumo da Suite

Total de testes	8 (4 em SCC_Testes + 4 em Dynatrace)
Framework	JUnit 5 (Jupiter) + REST Assured 5.x
Relatório	ExtentReports 5 — <code>target/SCC_Testes.html</code> e <code>target/Dynatrace.html</code>
Configuração	<code>src/test/resources/config.properties</code> (gitignore — credenciais e IPs internos)
Execução via Maven	<code>mvn test</code> — Surefire inclui apenas <code>SuiteInterno.java</code>
Java	17 (uso de pattern matching <code>instanceof Map<?,?> m</code> e <code>Set.of()</code>)

2. Arquitetura e Infraestrutura

2.1 Suite de Testes — SuiteInterno

A classe `br.com.testes.suite.SuiteInterno` é o ponto de entrada da suite. Usa `@Suite` e `@SelectClasses` do JUnit Platform para agrupar as classes na seguinte ordem:

Ordem	Classe	Qtd. testes
1	<code>SCC_Testes</code>	4
2	<code>Dynatrace</code>	4

O Maven Surefire está configurado para incluir somente `**/SuiteInterno.java`, garantindo que nenhuma outra classe de teste seja executada inadvertidamente.

2.2 Configuração — ConfigLoader

A classe `ConfigLoader` lê o arquivo `config.properties` do classpath e expõe o método estático `get(String chave)`. Se a propriedade não existir ou estiver vazia, uma exceção em tempo de execução é lançada — tornando o erro de configuração explícito.

Chaves usadas pelos testes:

`interno.base1` — URL base para senhas, filas e parâmetros de serviço

`interno.base2.pa` — URL base para o módulo PA (TAGs possíveis)

`interno.base2.fila` — URL base para o módulo de filas (TME)

2.3 ExtentReportExtension

Extensão JUnit que implementa os callbacks `BeforeAll`, `AfterAll`, `BeforeEach` e `AfterEach` para:

- ✓ Capturar output de cada teste via stream *tee* (grava no buffer E exibe no terminal simultaneamente)
- ✓ Registrar request e response com filtros globais do REST Assured via `replaceFiltersWith`
- ✓ Medir tempo de resposta HTTP e duração total do teste
- ✓ Gerar relatório HTML independente por classe (`SCC_Testes.html`, `Dynatrace.html`)
- ✓ Exibir estatísticas: total de testes, passaram, falharam, total de asserções, duração média
- ✓ Abrir o relatório automaticamente no Chrome ao fim de cada classe

Cada classe recebe sua própria instância de `ExtentReports` via `ConcurrentHashMap<String, ClassData>`, isolando relatórios por nome simples da classe.

3. SCC_Teste

Classe `br.com.testes.SCC_Teste`. Valida os endpoints de senha e TAGs do sistema de atendimento. O `@BeforeAll warmup()` desabilita reuso de instâncias HTTP e realiza uma requisição de aquecimento para estabilizar a conexão antes dos testes.

GET test_PA_ListarTAGsPossiveis

`{interno.base2.pa}/aspect/rest/PA/ListarTAGsPossiveis/5/2`

Descrição

Valida a listagem de grupos de TAGs e TAGs possíveis para a fila 5 da unidade 2. Verifica integridade referencial: cada TAG deve ter `GrupoId` apontando para um grupo existente na mesma resposta.

Parâmetros de path

Parâmetro	Tipo	Valor
<code>param1</code>	Integer	5 (fila)
<code>param2</code>	Integer	2 (unidade)

Validações

- ✓ Status HTTP `200`
- ✓ `result.GruposDeTags.Quantidade` confere com tamanho de `Itens`
- ✓ `result.TAGs.Quantidade` confere com tamanho de `Itens`
- ✓ Cada grupo: `ID` (Integer) e `Nome` (String não vazia)
- ✓ Cada TAG: campos `ID`, `Nome`, `GrupoId`, `Cor` presentes
- ✓ Cada TAG: `Cor` em hexadecimal de 6 dígitos (ex: `FF5733`)
- ✓ Integridade referencial: `GrupoId` de cada TAG existe na lista de grupos
- ✓ Campos novos na resposta: alerta , não falha o teste

POST**test_GerarSenha**`{interno.base1}/aspect/rest/senha/gerarsenha`**Descrição**

Valida geração de senha **sem prioridade** para a fila 5 da unidade 2. Senhas sem prioridade são compostas exclusivamente por dígitos numéricos.

Body

Campo	Tipo	Valor
Fila	String	"5"
Totem	String	"0"
IdUnidade	String	"2"

Validações

- ✓ Status HTTP `200`
- ✓ Content-Type contém `application/json`
- ✓ Tempo de resposta < 5.000 ms
- ✓ `result.sucesso` é `true` (Boolean)
- ✓ `result.senha`: apenas dígitos, valor > 0, entre 1 e 10 caracteres
- ✓ `result.emissao`: formato `MM/dd/yyyy HH:mm:ss`, data de hoje, recente (últimos 10 min)
- ✓ Campos exatos: `sucesso`, `senha`, `emissao` — sem extras nem faltando

Atenção: O campo `emissao` usa formato americano `MM/dd/yyyy` (mês/dia/ano), conforme retornado pela API. Comportamento intencional, documentado em `validarEmissao()`.

POST**test_GerarSenhaComPrioridade**`{interno.base1}/aspect/rest/senha/gerarsenha`**Descrição**

Valida geração de senha **com prioridade**. O formato é distinto: uma letra seguida de dígitos (ex: `T4147`).

Body

Campo	Tipo	Valor
<code>Fila</code>	Integer	1
<code>Totem</code>	Integer	0
<code>IdUnidade</code>	Integer	2
<code>Prioridade</code>	Integer	1

Validações

- ✓ Status HTTP `200`
- ✓ Content-Type contém `application/json`
- ✓ Tempo de resposta < 5.000 ms
- ✓ `result.sucesso` é `true` (Boolean)
- ✓ `result.senha`: regex `[A-Za-z]\d+`, parte numérica > 0, entre 2 e 10 caracteres
- ✓ `result.emissao`: mesmos critérios de `test_GerarSenha`
- ✓ Campos exatos: `sucesso`, `senha`, `emissao`

GET**test_ChamarProximo**`{interno.base1}/aspect/rest/senha/Chamarproximo/1/2`**Descrição**

Chama o próximo atendimento na fila 1 da unidade 2. Valida todos os campos retornados: senha chamada, data/hora, ID e nome da fila.

Parâmetros de path

Parâmetro	Tipo	Valor
<code>fila</code>	Integer	1
<code>unidade</code>	Integer	2

Validações

✓ Status HTTP `200`

✓ Content-Type contém `application/json`

✓ Tempo de resposta < 5.000 ms

✓ `result.Sucesso` é `true` (Boolean)

✓ `result.Senha`: apenas dígitos, valor > 0, entre 1 e 10 caracteres

✓ `result.DataHora`: formato `dd/MM/yyyy HH:mm:ss` (formato brasileiro)

✓ `result.IdFila`: Integer positivo (> 0)

✓ `result.NomeFila`: String não vazia

✓ Campos exatos: `Sucesso`, `Senha`, `DataHora`, `IdFila`, `NomeFila`

4. Dynatrace

Classe `br.com.testes.Dynatrace`. Valida endpoints de monitoramento de filas e parâmetros de configuração do serviço. A URL principal é carregada como constante estática (`API_URL`) no carregamento da classe. Cada teste usa `.log().ifValidationFails()` — logs aparecem somente em caso de falha.

GET test_ValidarParametrosServico

```
{interno.base1}/aspect/rest/Servico/Parametros/1
```

Descrição

Valida a estrutura completa de parâmetros do serviço: nome, IP e grupos de configuração (endpoint Dynatrace, endpoint SCC, API key).

Validações

- ✓ Status HTTP `200`
- ✓ `result.nomeServico`: String não vazia
- ✓ `result.enderecoIP`: formato IPv4 válido (`\d{1,3}.\d{1,3}.\d{1,3}.\d{1,3}`)
- ✓ `result.configuracoes`: lista não vazia
- ✓ Cada grupo: campos `grupo` (String) e `parametros` (lista não vazia)
- ✓ Cada parâmetro: `chave` e `valor` presentes e não vazios
- ✓ `EndPointDynatrace`: URL com `https://`
- ✓ `EndPointSccListarUnidade`: URL com `http://`
- ✓ `ApiKeyDynatrace`: presente e não vazia (valor não exibido no relatório)
- ✓ Campos novos em `result`: alerta, não falha

Delegação: Validação estrutural delegada a

```
Servico_Parametros.validarConfiguracoes()
```

GET**test_ValidarListarTMEDIA**

```
{interno.base2.fila}/aspect/rest/fila/ListarTMEDIA/?Unidade=1&Fila=1,2,3,4,5
```

Descrição

Lista o TME (Tempo Médio de Espera) do dia para as filas 1 a 5 da unidade 1. Retorna ID, nome e TME acumulado do dia por fila.

Parâmetros de query

Parâmetro	Valor
Unidade	1
Fila	"1,2,3,4,5"

Validações

✓ Status HTTP **200**

✓ `result.Filas`: lista não nula

✓ Cada fila: campos `ID`, `Nome`, `TMEDIA` presentes

✓ `ID` e `Nome`: String não vazia

✓ `TMEDIA`: formato `HH:mm:ss` — aceita horas > 23 (TME acumulado)

✓ Log individual por fila: ID, Nome e TMEDIA no relatório

✓ Relatório de campos novos para toda a lista

GET**test_ValidarMaiorEspera**

```
{interno.base1}/aspect/rest/fila/ObterMaiorEspera?Unidade=1&Fila=1,2,3,4,5
```

Descrição

Obtém a senha com maior tempo de espera entre as filas 1 a 5 da unidade 1. Retorna número, fila de origem, data de entrada e tempo de espera.

Validações

- ✓ Status HTTP **200**
- ✓ `result.Senha` : objeto não nulo
- ✓ `Senha.Numero` : String numérica, valor > 0, entre 1 e 10 caracteres
- ✓ `Senha.Fila` : String não vazia
- ✓ `Senha.DataEntradaFila` : formato `dd/MM/yyyy HH:mm:ss` (brasileiro)
- ✓ `Senha.TempoEspera` : formato `HH:mm:ss`
- ✓ Campos exatos em `result.Senha` : `Numero` , `Fila` , `DataEntradaFila` , `TempoEspera`

GET**test_ValidarObterQtdEsperaV2**

```
{interno.base1}/aspect/rest/fila/ObterQtdEsperaV2?Unidade=1&Fila=1,2,3,4,5
```

Descrição

Quantidade de senhas aguardando atendimento em cada fila. Retorna nome da fila e quantidade de espera por fila.

Validações

- ✓ Status HTTP **200**
- ✓ `result.Filas` : lista não nula
- ✓ Cada fila: campos `NomeFila` e `QtdEspera` presentes
- ✓ `NomeFila` : String não vazia
- ✓ `QtdEspera` : String composta apenas por dígitos (numérico retornado como String)
- ✓ Log individual por fila: nome e quantidade no relatório
- ✓ Relatório de campos novos para toda a lista

5. Classes Utilitárias

5.1 Metodos_Rest

Centraliza todas as validações reutilizadas pelos testes. Cada método realiza asserções JUnit e imprime o resultado no `System.out`, que é capturado pela `ExtentReportExtension` e exibido no relatório HTML.

Método	Categoria	Descrição
<code>validarBoolean(map, campo, esperado)</code>	Campo	Campo não nulo, tipo Boolean, igual ao valor esperado.
<code>validarString(map, campo)</code>	Campo	Campo não nulo, tipo String, não vazio.
<code>validarInteiro(map, campo)</code>	Campo	Campo não nulo e tipo Integer.
<code>validarInteiroPositivo(map, campo)</code>	Campo	Integer com valor > 0.
<code>validarContentType(response)</code>	HTTP	Content-Type contém <code>application/json</code> .
<code>validarTempoResposta(response)</code>	HTTP	Tempo de resposta < 5.000 ms (constante <code>RESPONSE_TIMEOUT_MS</code>).
<code>validarSenhaSemPrioridade(map, campo)</code>	Campo	String numérica pura: apenas dígitos, valor > 0, tamanho entre 1 e 10.
<code>validarSenhaComPrioridade(map, campo)</code>	Campo	1 letra + dígitos, parte numérica > 0, tamanho entre 2 e 10.
<code>validarEmissao(map, campo)</code>	Campo	Data <code>MM/dd/yyyy HH:mm:ss</code> (americano): hoje, não futura, recente (últimos 10 min).
<code>validarQuantidade(lista, qtd, label)</code>	Lista	Tamanho da lista confere com a quantidade declarada pela API.

<code>validarCamposItem(map, esperados, label)</code>	Estrutura	Todos os campos esperados presentes. Extras geram alerta, não falha.
<code>validarCamposExatos(map, esperados)</code>	Estrutura	Campos são <i>exatamente</i> os esperados — sem extras nem faltando.
<code>validarCorHex(map, campo)</code>	Campo	Hexadecimal de 6 dígitos (<code>[0-9A-Fa-f]{6}</code>).
<code>validarReferenciaExiste(lista, chave, valor, label)</code>	Referência	Integridade referencial: o valor existe na lista pelo campo chave.
<code>validarStringNumerica(map, campo)</code>	Campo	String contendo apenas dígitos.
<code>validarHorario(map, campo)</code>	Campo	Formato <code>HH:mm:ss</code> (aceita horas > 23 para TME acumulado).
<code>validarDataHoraBR(map, campo)</code>	Campo	Formato <code>dd/MM/yyyy HH:mm:ss</code> (brasileiro).
<code>validarIP(map, campo)</code>	Campo	Endereço IPv4 no formato <code>n.n.n.n</code> .
<code>validarURL(valor, protocolo)</code>	Campo	URL não vazia, iniciando com o protocolo informado.
<code>imprimirCabecalho(titulo, apiUrl)</code>	Relatório	Imprime cabeçalho de seção: título, URL testada, início das validações.
<code>relatorioCampos(map, esperados)</code>	Relatório	Exibe campos novos detectados; não falha.
<code>relatorioCamposLista(lista, esperados, rotulo)</code>	Relatório	Itera lista e exibe campos novos por item; não falha.
<code>validarChavesNovas(detectadas, baseline, rotulo)</code>	Baseline	Compara chaves com arquivo de baseline; adiciona novas automaticamente ao arquivo.

5.2 Servico_Parametros

Separa a lógica de validação de configurações do serviço da classe de teste, permitindo reuso em futuros testes que consumam o mesmo endpoint.

Método	Descrição
<code>validarConfiguracoes(configs, camposGrupo, camposParam)</code>	Itera cada grupo de configuração: valida campos estruturais, itera parâmetros e aplica validações específicas por chave conhecida (<code>EndPointDynatrace</code> , <code>EndPointScclistarUnidade</code> , <code>ApiKeyDynatrace</code>).